

Capitolo 1 — Condizionamento e stabilità

Appunti ragionati di Fondamenti di Calcolo Numerico

2026-07-30

1. Condizionamento e stabilità

Obiettivo

Un problema matematico viene spesso risolto numericamente dopo almeno tre trasformazioni:

1. il problema originario può essere sostituito da un'approssimazione finito-dimensionale;
2. i dati disponibili possono essere perturbati rispetto ai dati esatti;
3. l'algoritmo viene eseguito su una macchina che rappresenta i numeri con un numero finito di cifre.

Il risultato calcolato può quindi differire da quello matematicamente esatto per cause diverse, che devono essere separate prima di poterle controllare.

Idea chiave. Il capitolo distingue ciò che dipende **dal problema** da ciò che dipende **dall'algoritmo**. Il **condizionamento** misura la sensibilità del problema agli errori sui dati; la **stabilità** misura l'errore introdotto dall'algoritmo e dall'aritmetica finita.

Filo logico

```
dati perturbati
  |
  v
errore inerente -----> condizionamento del problema

aritmetica finita
  |
  v
errore algoritmico ---> stabilità dell'algoritmo

errore inerente + errore algoritmico
  |
  v
errore totale

stabilità + costo computazionale
  |
  v
scelta dell'algoritmo
```

1.1 Tipi di errore

Nel calcolo numerico si distinguono tre sorgenti principali di errore:

- **errore di approssimazione**, dovuto alla sostituzione di un problema infinito-dimensionale con un problema finito-dimensionale;
- **errore inerente**, dovuto alla perturbazione dei dati;
- **errore algoritmico**, dovuto all'esecuzione dell'algoritmo in aritmetica finita.

In questo capitolo l'attenzione è rivolta soprattutto agli ultimi due.

Errore assoluto ed errore relativo

Sia x un valore esatto e sia $\tilde{x}$ una sua perturbazione.

Definizione. L'errore assoluto è

$$\varepsilon = \tilde{x} - x.$$

Se $x \neq 0$, l'errore relativo è

$$\varepsilon_x = \frac{\tilde{x} - x}{x}.$$

Dalla definizione di errore relativo segue

$$\tilde{x} = x(1 + \varepsilon_x).$$

Questa forma sarà usata continuamente: una perturbazione viene rappresentata come il valore esatto moltiplicato per un fattore $1 + \varepsilon_x$.

Interpretazione. L'errore assoluto misura *quanto* cambia il valore; l'errore relativo misura quanto quel cambiamento pesa rispetto alla scala del dato.

Esempio: stesso errore assoluto, significato diverso

Se una festa prevista per 100 persone ne riceve 104,

$$\varepsilon = 4, \quad \varepsilon_x = \frac{4}{100} = 4\%.$$

Se una cena prevista per 10 persone ne riceve 14,

$$\varepsilon = 4, \quad \varepsilon_y = \frac{4}{10} = 40\%.$$

L'errore assoluto coincide, ma il significato della perturbazione è completamente diverso.

2. Errore inerente

Consideriamo una funzione

$$f : \mathbb{R}^n \rightarrow \mathbb{R},$$

un dato esatto

$$x = (x_1, \dots, x_n),$$

e un dato perturbato

$$\tilde{x} = (\tilde{x}_1, \dots, \tilde{x}_n), \quad \tilde{x}_i = x_i(1 + \varepsilon_i).$$

Supponiamo che gli errori relativi ε_i siano piccoli, così che i prodotti di due errori siano trascurabili rispetto ai termini di primo ordine.

Definizione. L'errore inerente è l'errore relativo sul risultato causato esclusivamente dagli errori sui dati:

$$\varepsilon_{\text{in}} = \frac{f(\tilde{x}) - f(x)}{f(x)}.$$

Il problema è **ben condizionato** se piccoli errori relativi sui dati producono un errore inerente basso; è **mal condizionato** quando tali errori vengono fortemente amplificati.

Attenzione. Il condizionamento è una proprietà del **problema**, non dell'algoritmo usato per risolverlo.

2.1 Stima al primo ordine dell'errore inerente

Teorema 1.4. Siano

$$x = (x_1, \dots, x_n) \in \mathbb{R}^n$$

e $f : \mathbb{R}^n \rightarrow \mathbb{R}$ due volte differenziabile in un intorno di x . Se

$$\tilde{x}_i = x_i(1 + \varepsilon_i)$$

e si trascurano gli errori di ordine superiore al primo, allora

$$\varepsilon_{\text{in}} \approx \sum_{i=1}^n \varepsilon_i \frac{x_i}{f(x)} \frac{\partial f(x)}{\partial x_i}.$$

Le quantità

$$c_i = \frac{x_i}{f(x)} \frac{\partial f(x)}{\partial x_i}$$

sono i **coefficienti di amplificazione**.

Dimostrazione

Poiché f è due volte differenziabile, possiamo sviluppare $f(\tilde{x})$ intorno a x :

$$f(\tilde{x}) = f(x) + \sum_{i=1}^n \frac{\partial f(x)}{\partial x_i} (\tilde{x}_i - x_i) + \frac{1}{2} \sum_{i,j=1}^n \frac{\partial^2 f(z)}{\partial x_i \partial x_j} (\tilde{x}_i - x_i)(\tilde{x}_j - x_j),$$

dove z appartiene al segmento che congiunge x e $\tilde{x}$.

Dalla rappresentazione relativa della perturbazione,

$$\tilde{x}_i = x_i(1 + \varepsilon_i),$$

segue

$$\tilde{x}_i - x_i = x_i \varepsilon_i.$$

Sostituendo nello sviluppo:

$$f(\tilde{x}) - f(x) = \sum_{i=1}^n \frac{\partial f(x)}{\partial x_i} x_i \varepsilon_i + \frac{1}{2} \sum_{i,j=1}^n \frac{\partial^2 f(z)}{\partial x_i \partial x_j} x_i x_j \varepsilon_i \varepsilon_j.$$

Il secondo addendo contiene prodotti $\varepsilon_i \varepsilon_j$, cioè termini di secondo ordine. Trascurandoli:

$$f(\tilde{x}) - f(x) \approx \sum_{i=1}^n \frac{\partial f(x)}{\partial x_i} x_i \varepsilon_i.$$

Dividendo per $f(x)$:

$$\varepsilon_{\text{in}} = \frac{f(\tilde{x}) - f(x)}{f(x)} \approx \sum_{i=1}^n \varepsilon_i \frac{x_i}{f(x)} \frac{\partial f(x)}{\partial x_i}.$$

Questo mostra anche perché i coefficienti c_i misurano l'amplificazione delle perturbazioni relative dei singoli dati.

Idea chiave. La sensibilità compare già nella derivata della funzione: non nasce dall'algoritmo. L'algoritmo può aggiungere altri errori, ma non può eliminare il cattivo condizionamento intrinseco del problema.

Caso scalare

Per $n = 1$,

$$\varepsilon_{\text{in}} \approx \varepsilon_x \frac{x f'(x)}{f(x)}.$$

2.2 Somma, prodotto e quoziente

Somma

Per

$$s(x, y) = x + y,$$

i coefficienti di amplificazione sono

$$\frac{x}{x+y}, \quad \frac{y}{x+y},$$

quindi

$$\varepsilon_{\text{in}}^{(s)} \approx \varepsilon_x \frac{x}{x+y} + \varepsilon_y \frac{y}{x+y}.$$

Se x e y hanno segno opposto e modulo simile, $x + y$ è piccolo e i coefficienti possono diventare molto grandi.

Definizione / fenomeno. Questa perdita di accuratezza è detta **cancellazione**: sottraendo quantità quasi uguali, le cifre significative comuni si eliminano e gli errori relativi preesistenti possono essere fortemente amplificati.

Prodotto

Per

$$p(x, y) = xy,$$

si ottiene

$$\varepsilon_{\text{in}}^{(p)} \approx \varepsilon_x + \varepsilon_y.$$

Quoziente

Per

$$q(x, y) = \frac{x}{y},$$

si ottiene

$$\boxed{\varepsilon_{\text{in}}^{(q)} \approx \varepsilon_x - \varepsilon_y}.$$

Prodotto e quoziente non presentano coefficienti di amplificazione che esplodono per effetto della sola combinazione degli operandi; la somma può invece diventare fortemente mal condizionata in prossimità della cancellazione.

3. Errore algoritmico

L'errore inerente esiste anche disponendo di un calcolo matematico ideale: nasce dalla perturbazione dei dati.

Quando il calcolo viene eseguito da un computer compare una seconda sorgente: i numeri reali devono essere rappresentati con un numero finito di cifre e ogni risultato intermedio deve essere nuovamente rappresentato nella macchina.

3.1 Aritmetica finita

Un computer può essere schematizzato mediante:

- memoria;
- unità di elaborazione;
- dispositivi di input/output.

Dal punto di vista numerico è essenziale che la memoria sia finita. Non è possibile rappresentare tutti i numeri reali con infinite cifre.

3.2 Richiamo: rappresentazione in base β

Una base β usa le cifre

$$0, 1, \dots, \beta - 1.$$

Un numero viene espresso come combinazione delle potenze della base.

In base 10,

$$2345.67 = 2 \cdot 10^3 + 3 \cdot 10^2 + 4 \cdot 10 + 5 + 6 \cdot 10^{-1} + 7 \cdot 10^{-2}.$$

In base 2,

$$10110.11_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2 + 0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 22.75_{10}.$$

Le regole aritmetiche sono le stesse; cambia la base rispetto alla quale vengono interpretate le cifre.

3.3 Rappresentazione floating point

Un numero reale positivo può essere scritto nella forma normalizzata

$$x = \beta^p \sum_{i=1}^{\infty} d_i \beta^{-i}, \quad 0 \leq d_i < \beta, \quad d_1 \neq 0.$$

Qui:

- p è l'esponente;
- d_i sono le cifre;
- la somma costituisce la mantissa.

Fissati:

- il numero di cifre t ;
- un limite inferiore $-N$ per l'esponente;
- un limite superiore M ;

si ottiene un insieme finito di **numeri di macchina**.

Se l'esponente è troppo piccolo si verifica **underflow**; se è troppo grande si verifica **overflow**.

3.4 Troncamento e arrotondamento

Definizione. Per

$$x = \beta^p \sum_{i=1}^{\infty} d_i \beta^{-i},$$

il **troncamento a t cifre** è

$$\text{trn}_t(x) = \beta^p \sum_{i=1}^t d_i \beta^{-i}.$$

Se β è pari, l'**arrotondamento a t cifre** conserva le prime t cifre e aggiunge un'unità nell'ultima posizione quando la cifra successiva soddisfa

$$d_{t+1} \geq \frac{\beta}{2}.$$

Esempio in base 10

Con $\beta = 10$ e $t = 3$:

$$321.2 = 0.3212 \cdot 10^3$$

produce

$$\text{trn}_3(x) = \text{arr}_3(x) = 0.321 \cdot 10^3.$$

Invece

$$321.7 = 0.3217 \cdot 10^3$$

produce

$$\text{trn}_3(x) = 0.321 \cdot 10^3, \quad \text{arr}_3(x) = 0.322 \cdot 10^3.$$

3.5 Errore di rappresentazione e precisione di macchina

Teorema 1.8. Fissata una base pari β e una rappresentazione con t cifre,

$$\left| \frac{\text{trn}_t(x) - x}{x} \right| < \beta^{1-t},$$

mentre

$$\left| \frac{\text{arr}_t(x) - x}{x} \right| < \frac{\beta^{1-t}}{2}.$$

Dimostrazione: maggiorazione della coda

Poiché $d_i \leq \beta - 1$,

$$\sum_{i=m+1}^{\infty} d_i \beta^{-i} \leq (\beta - 1) \sum_{i=m+1}^{\infty} \beta^{-i}.$$

Ponendo $k = i - (m + 1)$,

$$\sum_{i=m+1}^{\infty} \beta^{-i} = \beta^{-(m+1)} \sum_{k=0}^{\infty} \beta^{-k}.$$

La serie geometrica vale

$$\sum_{k=0}^{\infty} \beta^{-k} = \frac{1}{1 - \beta^{-1}} = \frac{\beta}{\beta - 1}.$$

Quindi

$$\boxed{\sum_{i=m+1}^{\infty} d_i \beta^{-i} \leq \beta^{-m}}.$$

Inoltre, essendo $d_1 \geq 1$,

$$x = \beta^p \sum_{i=1}^{\infty} d_i \beta^{-i} \geq \beta^p d_1 \beta^{-1} \geq \beta^{p-1}.$$

Caso del troncamento

Il troncamento elimina la coda dalla posizione $t + 1$, dunque

$$x - \text{trn}_t(x) = \beta^p \sum_{i=t+1}^{\infty} d_i \beta^{-i}.$$

Allora

$$\left| \frac{\text{trn}_t(x) - x}{x} \right| = \frac{x - \text{trn}_t(x)}{x} \leq \frac{\beta^p \sum_{i=t+1}^{\infty} d_i \beta^{-i}}{\beta^{p-1}}.$$

Quindi

$$\left| \frac{\text{trn}_t(x) - x}{x} \right| \leq \beta \sum_{i=t+1}^{\infty} d_i \beta^{-i} \leq \beta \beta^{-t} = \boxed{\beta^{1-t}}.$$

Caso dell'arrotondamento

Bisogna distinguere due casi.

Caso 1: $d_{t+1} < \beta/2$ Non viene aggiunta un'unità all'ultima cifra conservata. La coda trascurata inizia con una cifra che soddisfa

$$d_{t+1} \leq \frac{\beta}{2} - 1.$$

Separando la prima cifra della coda:

$$\sum_{i=t+1}^{\infty} d_i \beta^{-i} = d_{t+1} \beta^{-(t+1)} + \sum_{i=t+2}^{\infty} d_i \beta^{-i}.$$

Usando la maggiorazione precedente sulla parte restante si ottiene

$$\left| \frac{\text{arr}_t(x) - x}{x} \right| < \boxed{\frac{\beta^{1-t}}{2}}.$$

Caso 2: $d_{t+1} \geq \beta/2$ L'arrotondamento aggiunge β^{-t} alla mantissa troncata. L'errore è quindi la differenza fra questo incremento e la coda eliminata:

$$\text{arr}_t(x) - x = \beta^p \left(\beta^{-t} - \sum_{i=t+1}^{\infty} d_i \beta^{-i} \right).$$

Poiché la prima cifra eliminata è almeno $\beta/2$, la coda occupa almeno metà dell'unità aggiunta; la differenza residua è quindi minore di metà unità nell'ultima posizione:

$$\left| \frac{\text{arr}_t(x) - x}{x} \right| < \boxed{\frac{\beta^{1-t}}{2}}.$$

Orale — punto da saper ricostruire. La dimostrazione non consiste nel ricordare soltanto le due maggiorazioni finali: il passaggio chiave è stimare la coda della mantissa con una serie geometrica e usare $x \geq \beta^{p-1}$ per trasformare l'errore assoluto in errore relativo.

Definizione. La **precisione di macchina** è

$$\eta = \begin{cases} \beta^{1-t}, & \text{troncamento,} \\ \frac{\beta^{1-t}}{2}, & \text{arrotondamento.} \end{cases}$$

Ne segue che la rappresentazione floating point di un numero non soggetto a underflow o overflow soddisfa

$$\left| \frac{\text{fl}(x) - x}{x} \right| < \eta.$$

4. Definizione di errore algoritmico

Definizione. Dati una funzione

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

e dati di input che siano numeri di macchina, indichiamo con

$$\text{fl}(f(x))$$

il risultato ottenuto dal computer eseguendo uno specifico algoritmo.

L'errore algoritmico è

$$\varepsilon_{\text{alg}} = \frac{\text{fl}(f(x)) - f(x)}{f(x)}.$$

Un algoritmo è **stabile** se introduce un errore algoritmico basso.

Attenzione. Due algoritmi matematicamente equivalenti per il calcolo della stessa funzione possono avere stabilità completamente diversa.

4.1 Esempio: calcolo di e^{-30}

La serie

$$e^x = \sum_{i=0}^{\infty} \frac{x^i}{i!}$$

permette formalmente di calcolare e^{-30} in almeno due modi:

$$e^{-30} = \sum_{i=0}^{\infty} \frac{(-30)^i}{i!},$$

oppure

$$e^{-30} = \frac{1}{\sum_{i=0}^{\infty} \frac{30^i}{i!}}.$$

Le due espressioni sono matematicamente equivalenti, ma il primo calcolo somma termini alternati molto grandi che devono cancellarsi quasi completamente per ottenere un numero piccolissimo.

L'aritmetica finita rende questa cancellazione numericamente pericolosa.

Idea chiave. L'equivalenza algebrica non implica equivalenza numerica.

4.2 Errore di una singola operazione

Teorema 1.12. Siano a e b numeri di macchina e sia

$$\text{op} \in \{+, -, \cdot, /\}.$$

Allora

$$\text{fl}(a \text{ op } b) = (a \text{ op } b)(1 + \varepsilon), \quad |\varepsilon| \leq \eta.$$

Dimostrazione

Poiché a e b sono già rappresentabili esattamente nella macchina, l'unico nuovo errore nasce dalla rappresentazione del risultato dell'operazione.

Quindi

$$\text{fl}(a \text{ op } b) = \text{trn}_t(a \text{ op } b)$$

oppure

$$\text{fl}(a \text{ op } b) = \text{arr}_t(a \text{ op } b).$$

Applicando la stima dell'errore di rappresentazione al numero reale

$$a \text{ op } b,$$

si ha

$$\left| \frac{\text{fl}(a \text{ op } b) - (a \text{ op } b)}{a \text{ op } b} \right| \leq \eta.$$

Definendo quel rapporto come ε , otteniamo

$$\text{fl}(a \text{ op } b) = (a \text{ op } b)(1 + \varepsilon), \quad |\varepsilon| \leq \eta.$$

4.3 Propagazione dell'errore in un'operazione

Consideriamo

$$z_3 = z_1 \text{ op } z_2.$$

Se z_1 e z_2 sono già perturbati da errori totali

$$\varepsilon_1^{(\text{tot})}, \quad \varepsilon_2^{(\text{tot})},$$

il computer calcola

$$\text{fl}(z_3) = \left(z_1(1 + \varepsilon_1^{(\text{tot})}) \text{ op } z_2(1 + \varepsilon_2^{(\text{tot})}) \right) (1 + \varepsilon),$$

dove ε è l'errore introdotto dalla nuova operazione.

L'effetto degli errori degli operandi è l'errore inerente dell'operazione:

$$\varepsilon_{\text{in}}^{(\text{op})} \approx c_1 \varepsilon_1^{(\text{tot})} + c_2 \varepsilon_2^{(\text{tot})}.$$

Trascurando i prodotti di errori,

$$\boxed{\varepsilon_3^{(\text{tot})} \approx \varepsilon + c_1 \varepsilon_1^{(\text{tot})} + c_2 \varepsilon_2^{(\text{tot})}}.$$

Il grafo si legge a ritroso: per stimare l'errore totale di un nodo si somma l'errore introdotto localmente all'errore proveniente dai predecessori, pesato dai coefficienti di amplificazione.

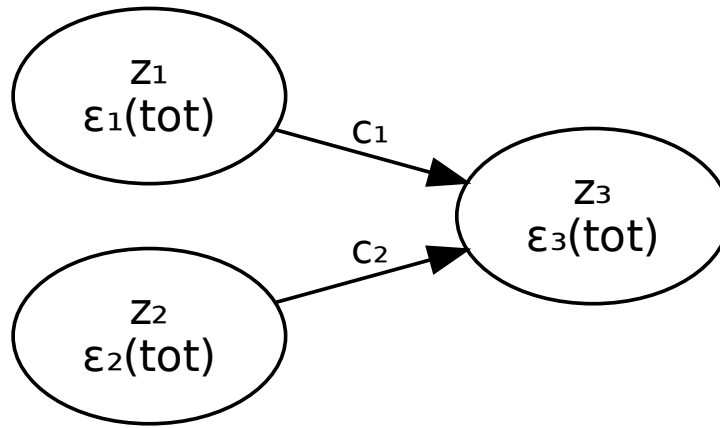


Figure 1: Propagazione dell'errore in una singola operazione

4.4 Due algoritmi per $(x + 1)^2$

Consideriamo

$$f(x) = (x + 1)^2 = x^2 + 2x + 1.$$

Primo algoritmo

Si calcola prima

$$z_1 = x + 1$$

e poi

$$z_2 = z_1^2.$$

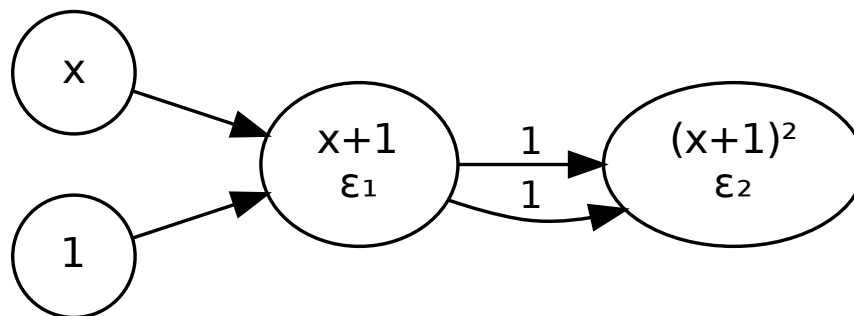


Figure 2: Primo algoritmo per il calcolo di $(x + 1)^2$

Per il prodotto finale entrambi i coefficienti di amplificazione sono 1. Si ottiene

$$\varepsilon_2^{(\text{tot})} = \varepsilon_2 + 2\varepsilon_1.$$

Poiché i coefficienti restano limitati, l'algoritmo è stabile.

Secondo algoritmo

Si calcola invece

$$x^2, \quad 2x, \quad x^2 + 2x, \quad x^2 + 2x + 1.$$

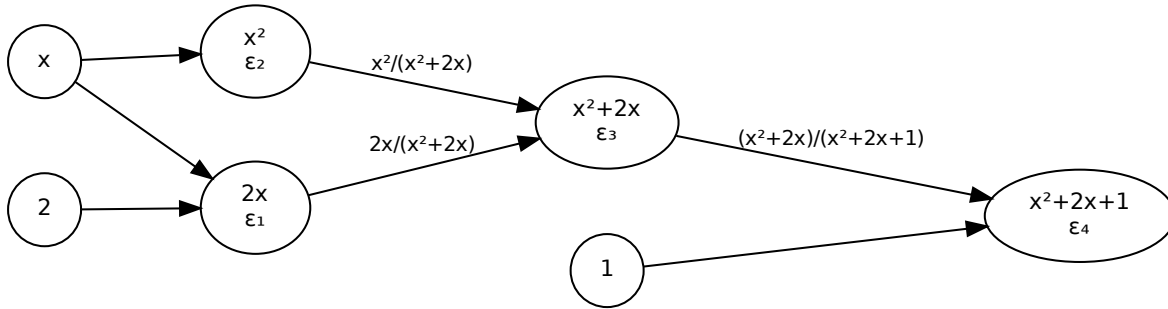


Figure 3: Secondo algoritmo per il calcolo di $x^2 + 2x + 1$

La propagazione porta a

$$\varepsilon_4^{(\text{tot})} = \varepsilon_4 + \frac{x^2 + 2x}{x^2 + 2x + 1} \varepsilon_3 + \frac{2x}{x^2 + 2x + 1} \varepsilon_1 + \frac{x^2}{x^2 + 2x + 1} \varepsilon_2.$$

Se

$$x^2 + 2x + 1 = (x + 1)^2 \approx 0,$$

cioè per $x \approx -1$, i coefficienti possono diventare molto grandi.

Il secondo algoritmo è quindi instabile in prossimità di $x = -1$.

Idea chiave. La funzione è la stessa; cambia il percorso computazionale. La stabilità appartiene all'algoritmo.

5. Errore totale

Consideriamo ora contemporaneamente:

- dati perturbati $\tilde{x}$;
- calcolo eseguito in aritmetica finita.

Definizione. L'errore totale è

$$\varepsilon_T = \frac{\text{fl}(f(\tilde{x})) - f(x)}{f(x)}.$$

5.1 Relazione fra errore inerente e algoritmico

Teorema 1.15.

$$\varepsilon_T = \varepsilon_{\text{in}} + \varepsilon_{\text{alg}} + \varepsilon_{\text{in}} \varepsilon_{\text{alg}}.$$

Al primo ordine,

$$\varepsilon_T \approx \varepsilon_{\text{in}} + \varepsilon_{\text{alg}}.$$

Dimostrazione

Partiamo dalla definizione:

$$\varepsilon_T = \frac{\text{fl}(f(\tilde{x})) - f(x)}{f(x)}.$$

Aggiungiamo e sottraiamo $f(\tilde{x})$:

$$\varepsilon_T = \frac{\text{fl}(f(\tilde{x})) - f(\tilde{x})}{f(x)} + \frac{f(\tilde{x}) - f(x)}{f(x)}.$$

Il secondo termine è esattamente

$$\varepsilon_{\text{in}}.$$

Nel primo termine moltiplichiamo e dividiamo per $f(\tilde{x})$:

$$\varepsilon_T = \frac{\text{fl}(f(\tilde{x})) - f(\tilde{x})}{f(\tilde{x})} \frac{f(\tilde{x})}{f(x)} + \varepsilon_{\text{in}}.$$

Il primo rapporto è ε_{alg} , mentre

$$\frac{f(\tilde{x})}{f(x)} = 1 + \varepsilon_{\text{in}}.$$

Quindi

$$\varepsilon_T = \varepsilon_{\text{alg}}(1 + \varepsilon_{\text{in}}) + \varepsilon_{\text{in}},$$

ossia

$$\varepsilon_T = \varepsilon_{\text{in}} + \varepsilon_{\text{alg}} + \varepsilon_{\text{in}}\varepsilon_{\text{alg}}.$$

Se gli errori sono piccoli, il prodotto dei due è di secondo ordine e viene trascurato:

$$\varepsilon_T \approx \varepsilon_{\text{in}} + \varepsilon_{\text{alg}}.$$

Orale — punto da saper ricostruire. L'errore totale separa nettamente due responsabilità: sensibilità del problema e qualità dell'algoritmo.

5.2 Esempio

Per

$$f(x, y) = \frac{x + y}{x^2},$$

il grafo permette di propagare insieme:

- gli errori relativi $\varepsilon_x, \varepsilon_y$ dei dati;
- gli errori di arrotondamento introdotti dalle operazioni.

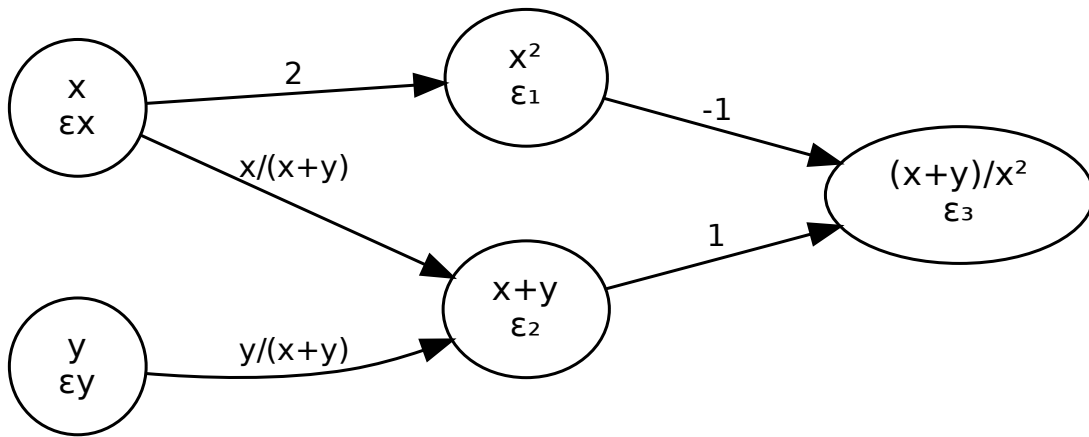


Figure 4: Grafo dell'errore totale per $f(x, y) = (x + y)/x^2$

Si ottiene una decomposizione della forma

$$\varepsilon_3^{(\text{tot})} = \underbrace{\varepsilon_3 + \varepsilon_2 - \varepsilon_1}_{\text{errore algoritmico}} + \underbrace{\left(-\varepsilon_x \frac{x + 2y}{x + y} + \varepsilon_y \frac{y}{x + y} \right)}_{\text{errore inerente}}.$$

L'algoritmo considerato è stabile, mentre il problema diventa mal condizionato quando

$$x \approx -y,$$

perché $x + y$ diventa piccolo.

6. Complessità computazionale

La stabilità non è l'unico criterio con cui confrontare algoritmi.

Definizione. Il **costo computazionale** è il numero di operazioni elementari richieste dall'algoritmo.

Nel modello adottato si contano soprattutto prodotti e divisioni, considerando le somme molto meno costose.

Il tempo di esecuzione può essere schematizzato come

$$\text{tempo} \approx \text{numero di operazioni} \times \text{tempo medio per operazione}.$$

Esempio: sistemi lineari

Per un sistema lineare quadrato di ordine n :

- la regola di Cramer richiede un numero di operazioni dell'ordine di $(n + 1)!$;
- l'eliminazione di Gauss richiede un numero di operazioni dell'ordine di

$$\frac{n^3}{3}.$$

La differenza di crescita rende Cramer impraticabile già per dimensioni moderate, anche se dal punto di vista matematico fornisce una formula esatta.

Idea chiave. Un algoritmo numerico deve essere valutato almeno su due assi distinti:

1. qualità numerica — stabilità;
2. costo — complessità computazionale.

7. Quadro riassuntivo

perturbazione dei dati $\longrightarrow \varepsilon_{\text{in}} \longrightarrow$ condizionamento del problema, aritmetica finita + algoritmo $\longrightarrow \varepsilon_{\text{alg}} \longrightarrow$ stabilità dell'algoritmo, $\varepsilon_T \approx \varepsilon_{\text{in}} + \varepsilon_{\text{alg}}.$

Il capitolo prepara così il linguaggio che verrà usato nei successivi problemi numerici: non basta trovare un metodo matematicamente corretto; occorre capire quanto il problema amplifichi gli errori, quanto l'algoritmo ne introduca di nuovi e quale costo sia necessario per ottenere il risultato.